

Multivariate Time Series Anomaly Detection Using Learnable Spatial-Temporal Graph Ordinary Differential Equations Network

Shiming He[✉], Qingqing Guo[✉], Keyao Feng, Diqing Liang[✉], Kun Xie[✉], *Member, IEEE*,
and Pradip Kumar Sharma[✉], *Senior Member, IEEE*

Abstract—Multivariate time series anomaly detection (MTSAD) plays a critical role in the Internet of Things (IoT) by identifying malfunctions and attacks. Graph Neural Networks (GNNs) have been widely employed in MTSAD to capture spatial features but require predefined and explicit graph structures. Graph Structure Learning (GSL) addresses this limitation by jointly learning the graph structure and downstream tasks. However, existing GSL-based MTSAD methods fail to effectively leverage prior knowledge and struggle with insufficient GNN depth, limiting their ability to capture long-range dependencies. To address these challenges, we propose a multivariate time series anomaly detection method based on a learnable spatio-temporal graph ordinary differential equation network (STGODE), named MAD-ODE. Our approach leverages hybrid graph learning, which includes two types of graph structures: a static similarity graph and a learnable graph. The static similarity graph is constructed using prior knowledge and provides a stable, interpretable representation of sensor dependencies. In contrast, the learnable graph captures complex relationships between sensors by optimizing its structure through backpropagation. This hybrid graph learning effectively incorporates both prior knowledge and learned dependencies, ensuring robust and flexible modeling of sensor relationships. Furthermore, we design a STGODE predictor, which operates on both graph structures and employs continuous graph convolutional networks, enabling it to capturing long-range spatio-temporal dependencies for forecasting the next timestamp. Extensive experiments conducted on five datasets demonstrate that MAD-ODE achieves the best average performance and maintains stable results compared to existing methods.

Index Terms—Multivariate time series, graph neural networks, IoT security, anomaly detection, spatial-temporal graph ordinary differential equation network, graph structure learning.

I. INTRODUCTION

IN THE Internet of Things (IoT), attacks can cause physical damages to life-dependent physical processes. IoT attack detection approaches typically analyze three primary data sources: sensor data [1], network traffic [2], [3], [4], and Supervisory Control and Data Acquisition (SCADA) execution context [5]. Among these, sensor data directly reflects deviations in observed physical states from expected behaviors, making it a critical resource for detecting anomalies. Various sensors continuously generate large volumes of time series data to monitor the critical infrastructures. This data, collected from multiple sensors, is referred to as multivariate time series (MTS). Abnormal changes in MTS often indicate not only system attacks but also equipment failures. If these anomalies are not detected and mitigated promptly, they can lead to catastrophic system failures, resulting in significant economic losses [6], [7]. Given the importance of timely anomaly detection, this paper focuses on sensor data, as it offers a direct and reliable means of monitoring deviations in physical processes.

For example, the Secure Water Treatment (SWaT) testbed consists of six interconnected processes (P1–P6): raw water supply and storage, chemical dosing, ultrafiltration (UF), dechlorination, reverse osmosis (RO), and RO permeate transfer and UF backwash, as shown in Fig. 1. Each process is equipped with several sensors, which are named based on their corresponding functions and processes. For instance, the sensor AIT-202 monitors the level of contaminants in the water. Initially, the value of AIT-202 exceeded 7.05. However, at 12:04:10 on 28 June 2015, an attacker manipulated the sensor value and set it to 6. This malicious action caused the chemical dosing pump P-203 to shut down, thereby disrupting the chemical dosing process and adversely impacting water quality. Therefore, multivariate time series anomaly detection (MTSAD) plays an essential role in ensuring the reliability and security of IoT systems [8].

In IoT systems, sensors are interdependent and interact with one another. Recently, graph neural networks (GNNs) [9] have been introduced into MTSAD to capture the relationships

Received 20 March 2025; revised 23 November 2025; accepted 30 November 2025. Date of publication 4 December 2025; date of current version 12 March 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62025201 and Grant 62272062, in part by the Science and Technology Innovation Program of Hunan Province under Grant 2023RC3139, and in part by the Natural Science Foundation of Hunan Province under Grant 2025JJ50373. (*Corresponding author: Kun Xie.*)

Shiming He, Qingqing Guo, Keyao Feng, and Diqing Liang are with the School of Computer Science and Technology and the Hunan Provincial Key Laboratory of Intelligent Processing of Big Data on Transportation, Changsha University of Science and Technology, Changsha 410114, China (e-mail: smhe_cs@csust.edu.cn; guoqingqing@stu.csust.edu.cn; kyfeng@stu.csust.edu.cn).

Kun Xie is with the College of Computer Science and Electronics Engineering, the Ministry of Education Key Laboratory of “Fusion Computing of Supercomputing and Artificial Intelligence,” Hunan University, Changsha 410082, China (e-mail: xiekun@hnu.edu.cn).

Pradip Kumar Sharma is with the Department of Computing Science, University of Aberdeen, AB24 3FX Aberdeen, U.K. (e-mail: Pradip.sharma@abdn.ac.uk).

Digital Object Identifier 10.1109/TDSC.2025.3640165

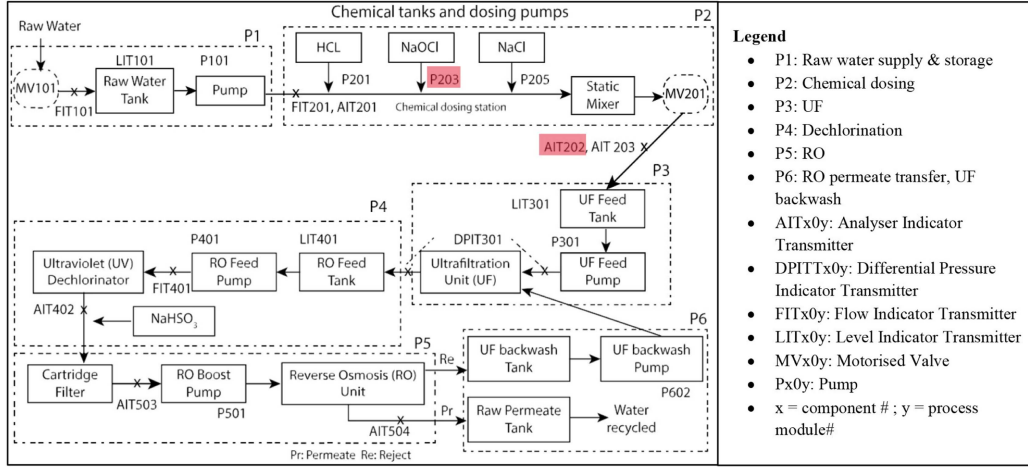


Fig. 1. Example of an IoT anomaly on a Water Treatment System. The Secure Water Treatment (SWaT) testbed consists of six interconnected processes (P1–P6). Each process is equipped with several sensors, which are named based on their corresponding functions and processes. For instance, the sensor AIT-202 monitors the level of contaminants in the water. Initially, the value of AIT-202 exceeded 7.05. However, at 12:04:10 on 28 June 2015, an attacker manipulated the sensor value and set it to 6. This malicious action caused the chemical dosing pump P-203 to shut down, thereby disrupting the chemical dosing process and adversely impacting water quality.

between sensors. However, GNNs require an explicit graph structure, which is typically constructed using domain knowledge. Existing GNN-based anomaly detection methods, such as MTAD-TF [10], MTAD-GAT [11], Arvalus [12], MDGAT [13], DVGCRN [14], and GReLeN [15], rely on fixed graph structures designed by experts. Unfortunately, in real-world scenarios, sensor dependencies or graph structures are often hidden, difficult to infer, or expensive to acquire.

Graph Structure Learning (GSL) [16] offers a promising solution for these challenges when expert knowledge is unavailable, sensor dependencies are unknown, or access costs are prohibitive [17]. GSL learns an optimal graph structure to capture the dependencies between sensors and align with downstream anomaly detection tasks. Although GSL-based methods have shown potential in learning sensor relationships [18], existing GSL-based MTSAD methods [19], [20], [21], [22], [23], [24], [25], [26], [27], [28], [29], [30], [31], [32], [33] face the following challenges:

- *Existing methods focus solely on graph structure learning and fail to utilize partial known information:* Current GSL-based anomaly detection methods primarily rely on unknown graph structures during the learning process, ignoring the potential benefits of incorporating partially known information. As a result, these methods engage in a largely blind learning process, leading to suboptimal outcomes.
- *Existing methods suffer from insufficient depth in graph neural networks, limiting their ability to capture long-distance dependencies between sensors:* Most GSL-based anomaly detection methods employ spatio-temporal prediction models that separately model spatial and temporal patterns without accounting for their interactions, which significantly limits their representational capacity. Furthermore, Graph Convolutional Networks (GCNs) struggle to achieve deep

node representations by merely increasing the number of layers. Practical evidence shows that GCNs typically achieve their best performance with just two layers, as deeper GCNs suffer from the over-smoothing problem. Over-smoothing causes all node representations to converge to the same value, rendering deeper GCNs ineffective. These limitations severely restrict the depth of GCNs, making it challenging to capture rich spatial features and long-distance dependencies, ultimately hindering anomaly detection performance.

To address these challenges, we propose MAD-ODE, a novel framework to enhance MTSAD by dynamically learning graph structures and modeling long-distance dependencies using spatio-temporal GCNs based on ordinary differential equations (ODEs). The key contributions of MAD-ODE are summarized as follows:

- *Enhanced representation through hybrid graph learning:* To better capture and enrich the dependencies between sensors, we exploit two kinds of graph structures: a static, knowledge-driven similarity graph and a fully parameterized learnable graph. This hybrid graph learning significantly enhances the model's representational capacity, leveraging both learned and prior knowledge.
- *Capturing long-distance spatio-temporal dependencies:* We introduce spatio-temporal graph ODE network (STGODE) instead of discrete GCNs, which operates on both the static similarity graph and the learned graph for prediction and anomaly detection. This approach effectively captures long-distance spatio-temporal dependencies with continuous-time model, improving the model's resilience and enabling more accurate and robust multi-variate time series predictions.
- *Extensive evaluation on real-world datasets:* We conducted comprehensive experiments on five publicly available real-world datasets. MAD-ODE achieves the best

average performance and maintains stable results compared to existing methods. This underscores the robustness and reliability of MAD-ODE across diverse datasets and anomaly detection tasks. The code of this work is publicly available at <https://github.com/AIOps-tech/MAD-ODE>.

The rest sections of this paper are arranged as follows. Section II provides an overview of the related work. Section III introduces some preliminary knowledge and defines the problem. Section IV explains our proposed approach in detail, including a general overview and the functionality of individual components. In Section V, we analyze the performance and efficiency of the model through extensive experiments. Finally, our work is concluded with a summary of our findings and a discussion on future work in Section VI.

II. RELATED WORK

In addition to MTS sensor data, network traffic and SCADA execution context are also utilized to enhance the safety and security of IoT systems. For instance, Kitsune [2], EULER [3], and HyperVision [4] focus on network traffic and encrypted traffic analysis for intrusion detection in IoT environments. These methods have demonstrated high effectiveness in traffic-based anomaly detection. On the other hand, SCAPHY [5] combines SCADA execution context with physical sensor states to address security issues in industrial control and cyber-physical systems. By integrating these complementary data sources, SCAPHY provides a comprehensive approach to detecting and mitigating threats in complex industrial environments.

Research on MTSAD can be categorized into three principal classes. The first class primarily utilizes temporal features to detect anomalies, whereas the second class focuses on the spatial features of MTS. The third class addresses anomaly detection in cases where the spatial relationships are unknown.

A. Temporal Feature-Based Anomaly Detection

Temporal feature-based methods focus on capturing sequential patterns in time series data. Among these, Long Short-Term Memory (LSTM) networks have been widely adopted due to their ability to model temporal dependencies. For example, LSTM-NDT [34] leverages LSTM to achieve superior predictive performance while maintaining system interpretability by employing a nonparametric, dynamic, and unsupervised thresholding technique for anomaly detection. In contrast, MSCRED [35] constructs a multi-scale feature matrix and uses conv-LSTM neural architectures to encode inter-sensor correlations, enabling anomaly detection and diagnosis via a residual feature matrix. Additionally, MAD-GAN [36] employs LSTM as the backbone within a Generative Adversarial Network (GAN) framework. LSTM-DAE [37] proposes a two-stage framework, first identifying outlier candidates using operational cycle signals and then analyzing temporal coherence in sensor data to detect abnormalities. In contrast, LSTM-VAE [38] incorporates multi-modal observations and temporal dependencies into a latent space, reconstructs distributions, and evaluates anomalies using reconstruction-based scores.

Beyond LSTM, other recurrent neural networks (RNNs) are also applied in MTSAD. For example, OmniAnomaly [39] aims to learn the normal representation of time series data, and then calculate reconstruction errors to effectively identify anomalous points that deviate from the normal patterns, while DAGMM [40] combines a deep self-encoding mechanism with a Gaussian Mixture Model (GMM) to generate low-dimensional representations and reconstruction errors for anomaly detection.

Neural ordinary differential equations (ODEs) [41] define a vector field, enabling the continuous transformation of hidden states. As an emerging family of neural networks, Neural ODEs have demonstrated significant potential in modeling the dynamics of temporal data. Motivated by this, models such as LSTM-ODE and latent-ODE [42] integrate the strengths of LSTM, VAE, and Neural ODEs to effectively detect anomalies in MTS data, addressing challenges posed by sparsity and irregularity.

Recently, diffusion models have gained attention in MTS anomaly detection due to their generative capabilities. For instance, MadSGM [43] utilizes the probability flow ODEs of score-based generative models to synthesize samples and identify anomalies. Similarly, TimeADDM [44], a diffusion-based anomaly detection framework, accelerates the reverse denoising process by employing an ODE solver, efficiently recovering latent features from noise to enhance anomaly detection performance.

While these methodologies are rooted in temporal features, they often do not take account of the intricate correlations inherent in time series data [19].

B. Spatial Feature-Based Anomaly Detection

Spatial feature-based methods leverage the relationships between sensors to enhance anomaly detection. GNNs, particularly Graph Attention Networks (GATs), are pivotal for extracting features from graph-structured data, capturing both inter-sensor relationships and time-varying dependencies. For instance, MTAD-TF [10] combines multi-scale convolution networks with GAT to model temporal and spatial patterns, while MTAD-GAT [11] leverages GAT to uncover correlations among univariate time series and their temporal dependencies, integrating prediction and reconstruction for anomaly detection. Similarly, Arvalus [12] models system components as nodes and their interdependencies as edges, improving anomaly detection precision. MDGAT [13] employs multi-head dynamic attention to capture sensor dependencies across temporal and spatial domains. DVGCRN [14] focuses on uncertain interrelations in MTS data using reconstruction-based anomaly detection. Lastly, GReLeN [15] utilizes Variational Autoencoders (VAE) and Graph Convolutional Networks (GCN) to learn relationships in MTS data, reconstruct graphs, and detect anomalies.

Although GNNs are effective in modeling spatial relationships, they rely on predefined graph structures, which are often unavailable or expensive to obtain in real-world scenarios.

C. Unknown Spatial Relationship-Based Anomaly Detection

When explicit spatial relationships are unavailable, GSL provides a solution by learning the underlying graph structure

directly from the data. GSL-based methods can be broadly categorized into node feature-based similarity approaches and fully parameterized approaches [45].

Node feature-based similarity approaches model graph structures by computing node-wise similarity using metric learning functions, such as cosine similarity or dot product. These methods rely on the inherent features of nodes to measure their pairwise relationships. GDN [19] learns inter-sensor relationships based on adaptive node features to detect anomalies from expected distributions. HGT MAD [20] integrates GCNs and transformers to extract spatial and temporal features, while FuSAGNet [21] models interrelations using a fused sparse self-encoder graph for anomaly predictions and reconstructions. TS-GAT [22] leverages prior data knowledge to constrain graph learning, and methods like MEGA [23] and DPGLAD [24] adopt unidirectional graph learning. Additionally, GSC-MAD [25] employs graph updating with smooth and sparse constraints, while ACGSL [26] creates dynamic graph representations through subsequence-based learning.

In contrast, fully parameterized methods treat each element of the adjacency matrix as a learnable parameter. This parameterization allows the model to directly optimize the graph structure, providing greater flexibility to capture complex and latent relationships within the graph. GTA [27] use transformers for predictions, identifying anomalies by evaluating the disparity between predicted and actual values. GLAD [28] combines GATs with transformers to extract global and local features, MGCLAD learns inter- and intra-signal correlations using directed graphs, and MEGLAD [29] employs multiple GSL variations for diverse correlations. Lastly, GSLAD [33] uses diffusion graph convolution networks to detect anomalies.

Node feature-based similarity methods often rely on simple K-nearest neighbors (KNN) graphs with degree constraints, whereas fully parameterized GSL methods provide greater flexibility. However, transformer-based methods such as GTA, GLAD, and MGCLAD significantly increase computational complexity, which can be a limiting factor in large-scale applications.

III. THREAT MODEL AND PROBLEM DEFINITION

This paper focuses on anomaly detection in MTS of sensor data. Below, we first give the threat model and define the fundamental concepts of MTS, graph structure learning and hybrid graph learning-based MTSAD.

A. Threat Model

We adopt a threat model similar to those used in existing works on MTS of sensor data in industrial IoT, such smart water plant. In this model, the attacker can maliciously manipulate critical components such as the motorized valve, flow indication transmitter, pump, and consumers, or directly modify sensor readings. These attacks can lead to severe physical consequences, including overflow of the primary tank, increased chemical levels in the water, manipulation of tank draining and filling speeds, disruption of water supply to consumers, contamination-induced drainage of the primary grid, water leakage and pipe bursts,

intermittent water supply, and halting of chemical dosing to raw water. For example, an attacker may target the elevated reservoir (e.g., 2_LT_002) by manipulating the tank's draining and filling speeds. Additionally, the attacker could alter the readings of water quality sensors (e.g., 1_AIT_001), causing the raw water tank to drain unnecessarily. These coordinated actions can disrupt the normal operation of the system and compromise its safety and reliability.

We assume that adversaries cannot manipulate their behavior to evade detection, as the primary target of these attacks involves physical processes. Unlike purely cyber attacks, physical attacks inherently produce observable changes in sensor data, making them detectable when effective anomaly detection mechanisms are in place. This assumption simplifies the threat model by focusing on identifying deviations in physical system behaviors caused by malicious activities. The sensor data collected for training represents benign states without the presence of attacks. This ensures that the model learns normal system behavior and can effectively detect anomalies caused by malicious activities.

B. Multivariate Time Series

Multivariate time series data consists of a large number of regularly spaced and uninterrupted samples, which are characterized by N indicators and M timestamps. It is denoted by $X = (x_1, x_2, \dots, x_M)^T \in R^{M \times N}$. The i -th indicator is defined as $x^i = (x_1^i, x_2^i, \dots, x_M^i)$. The t -th timestamp encompasses N indicator values, which is defined as $x_t = (x_t^1, x_t^2, \dots, x_t^N)^T$. The history window on timestamp t with length ω is defined as a subsequence $X_t = (x_{t-\omega}, x_{t-\omega+1}, \dots, x_{t-1})^T \in R^{\omega \times N}$.

For anomaly detection, when the label y_t is 1, it means that there is an anomaly at timestamp t . When the label y_t is 0, it means that timestamp t is normal.

C. Graph Structure Learning

For a given multivariate time series $X \in R^{M \times N}$, the purpose of graph structure learning is to construct a graph $G = (V, E)$ and its corresponding graph topology or adjacency matrix $A^L \in R^{N \times N}$. Nodes (V) in the graph are the sensors that produce the indicators, and the hidden relationships between sensors are considered as edges (E).

$$A^L = \text{GSL}(X), \quad (1)$$

where $\text{GSL}()$ indicates graph structure learning function. The binary adjacency matrix (A^L) stores the edge information in the graph. $A_{i,j}^L$ is 1, which indicates an edge between node i and node j . On the contrary, $A_{i,j}^L$ is 0, which indicates no edge between node i and node j .

The constructed graph is subsequently fed into the downstream anomaly detection model, enabling a joint update of parameters in both the detection model and the graph structure learning model. Through this process, the graph structure is iteratively refined. This iterative parameter update scheme ensures that the graph structure progressively adapts to the specific requirements of the downstream anomaly detection task.

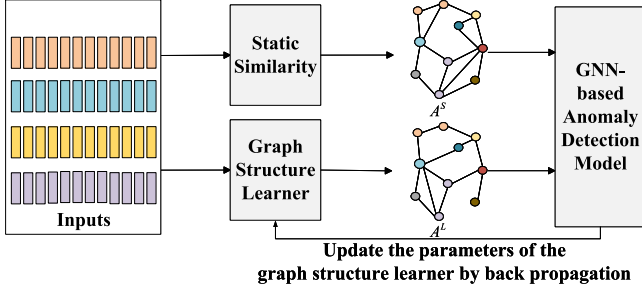


Fig. 2. The framework of hybrid graph learning-based multivariate time series anomaly detection.

D. Hybrid Graph Learning-Based Multivariate Time Series Anomaly Detection

Hybrid graph learning-based MSTAD operates in a semi-supervised manner (only normal time series instances are given in the training set). Hybrid graph learning incorporates two types of graph structures: the static similarity graph (A^S) and the learnable graph (A^L), as shown in Fig. 2. The static similarity graph A^S is constructed based on similarities derived from the raw data, providing a stable and interpretable representation of relationships among nodes. In contrast, the learnable graph A^L is generated through GSL, enabling the model to adaptively capture complex dependencies. Given the historical subsequence X_t , the model leverages both graph structures to predict the subsequent value $\hat{x}_t$. The prediction error between the actual value x_t and the predicted value $\hat{x}_t$ is treated as the anomaly score, denoted as s_t . An anomaly is deemed to have occurred at timestamp t if the anomaly score s_t exceeds a predefined threshold T .

The process can be formally described as follows:

$$A^S = \text{Similarity}(X), \quad (2)$$

$$\hat{x}_t = f(X_t, A^S, A^L), \quad (3)$$

$$s_t = \varphi(x_t, \hat{x}_t), \quad (4)$$

$$\hat{y}_t = \begin{cases} 1, & \text{if } s_t > T, \\ 0, & \text{if } s_t \leq T, \end{cases} \quad (5)$$

where $\text{Similarity}()$ indicates the similarity function, $f()$ denotes the prediction function, $\hat{x}_t$ represents the prediction value of x_t at timestamp t , φ is the function used to calculate the anomaly score, and T stands for the threshold.

This hybrid approach effectively combines prior knowledge (captured by A^S) and dynamically learned relationships (captured by A^L) to enhance the detection of anomalies in complex multivariate time series. By leveraging these complementary graph structures and prediction task, the MSTAD framework achieves robust and accurate anomaly detection without requiring labeled data.

IV. PROPOSED METHODOLOGY

A. Overview

To enrich and exploit the dependencies between sensors while capturing long-distance spatio-temporal relationships, we propose MAD-ODE.

MAD-ODE exploits hybrid graph learning, integrating a fully parameterized learnable graph with a static, knowledge-driven similarity graph. This hybrid graph learning enhances the model's representational capacity by leveraging both learned and prior knowledge. Consequently, MAD-ODE comprises two key graph components: **the static similarity graph structure generator** and **the graph structure learner**.

To effectively model long-distance spatio-temporal dependencies, MAD-ODE employs a continuous-time model STGODE as the predictor, replacing traditional discrete GCNs. STGODE operates on both the static similarity graph and the learned graph and to predict future values. The prediction errors are then used as anomaly scores.

MAD-ODE consists of four key modules: the static similarity graph structure generator, the graph structure learner, the STGODE [46] predictor, and the anomaly score and threshold selection module, as illustrated in Fig. 3.

- *Static similarity graph structure generator*: This module generates a static graph structure based on cosine similarity between nodes (indicators), denoted as A^{COS} . It calculates the cosine distance between indicators and selects the top-k most similar nodes as neighbors for each node. The fixed cosine graph structure provides a stable and complementary graph structure to the learned graph structure, enhancing the model's robustness and improving the accuracy of anomaly detection.
- *Graph structure learner*: This module generates a learnable graph structure, denoted as A^{FPM} . Firstly, the fully parameterized graph structure learner treats each element of the adjacency matrix as an independent and learnable parameter. In addition, the graph learning process is guided by a dynamic time warping (DTW) distance [47] prior graph. The DTW prior graph effectively measures the similarity between time series by capturing delay correlations and ensuring invariance to time shifts.
- *STGODE predictor*: This module predicts the next timestamp value in the time series with the static graph A^{COS} and the learned graph A^{FPM} . It uses a continuous GCN with residual connections to address the issue of excessive smoothing in traditional GCNs, which can cause node representations to converge to identical values. The predictor takes sub-sequences of the multivariate time series (generated using a sliding window), and the adjacency matrices of the static graph and the learned graph as input to predict the value at the next timestamp.
- *Anomaly scores and threshold selection*: This module identifies anomalies based on the prediction error. Anomaly scores are derived from the prediction error between the predicted values and the ground truth. A higher anomaly score indicates a higher likelihood of an anomaly. A threshold is selected to distinguish between normal and

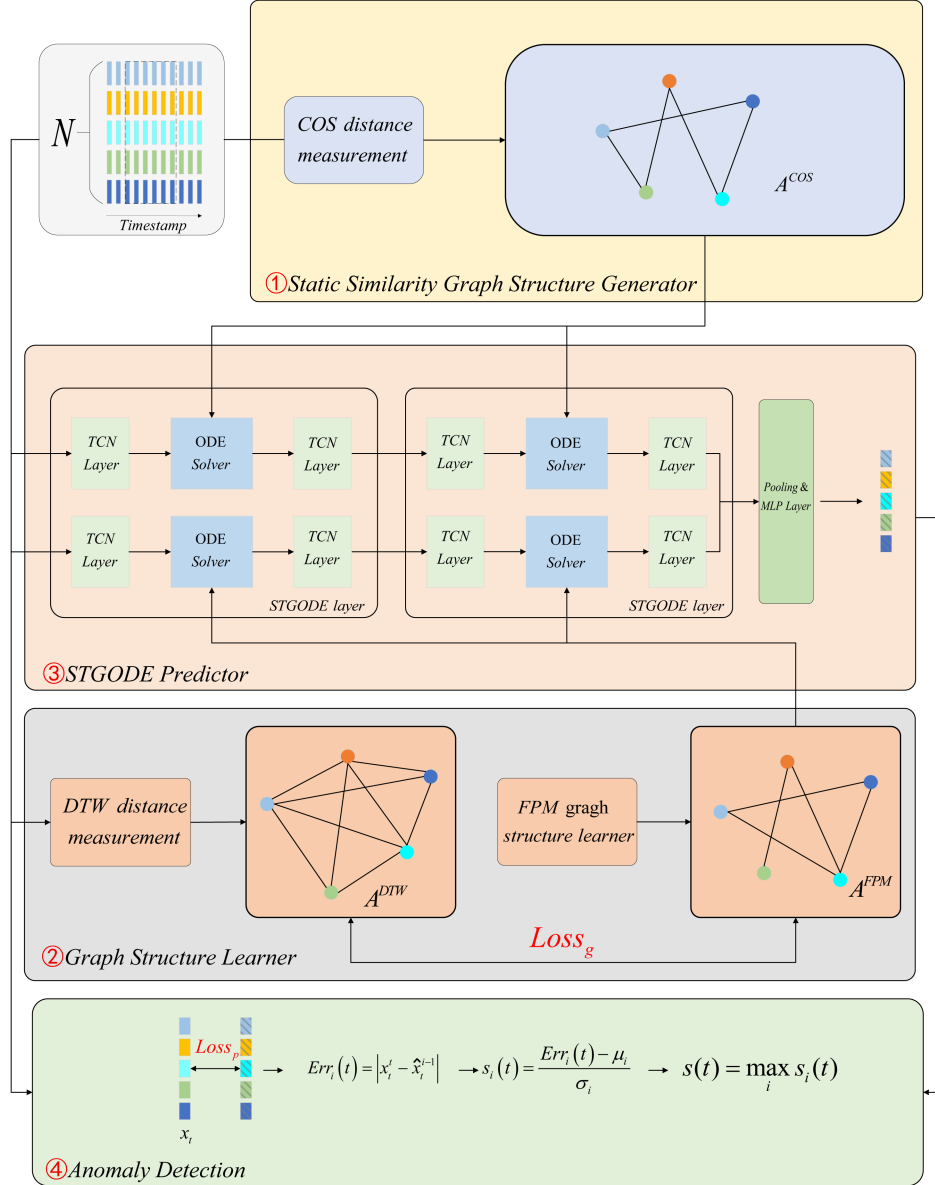


Fig. 3. The framework of MAD-ODE. MAD-ODE consists of four key modules: the static similarity graph structure generator, the STGODE predictor, and the anomaly score and threshold selection module. MAD-ODE exploits hybrid graph learning, which integrates a fully parameterized learnable graph with a static, knowledge-driven similarity graph, enhancing the model's representational capacity. MAD-ODE employs STGODE as the predictor to effectively model long-distance spatio-temporal dependencies. STGODE operates on both the learned graph and the static similarity graph to predict future values. The prediction errors are then used as anomaly scores.

anomalous sub-sequences. If the anomaly score exceeds the threshold, the prediction timestamp is classified as an anomaly; otherwise, it is considered normal.

In the following subsections, we give the details of each components.

B. Static Similarity Graph Structure Generator

The purpose of this module is to leverage known information from the raw time series data to construct a static graph structure A^{COS} . This graph captures the relationships (similarities) between sensors, which is essential for downstream tasks such as anomaly detection. It calculates the similarity between sensors

using cosine distance. The formula for cosine similarity is:

$$\cos(x^i, x^j) = \frac{x^i \bullet x^j}{\|x^i\| \bullet \|x^j\|}, \quad (6)$$

where $\cos(\cdot)$ denotes cosine distance calculation function, $\bullet$ denotes dot product, $\|\cdot\|$ denotes magnitude, and x^i denotes i -th sensor values.

For each sensor i , the module selects the top k most similar sensors as its neighbors. k can adjust the sparsity of the static graph's adjacency matrix and reduce the training cost.

$$A_{i,j}^{COS} = 1, j \in \text{topk}(\cos(x^i, x^j)), \quad (7)$$

where $A_{i,j}^{COS}$ represents the connection between sensors i and j , and $\text{topk}()$ selects the top k most similar sensors for each sensor i .

The static similarity graph structure offers a stable and interpretable representation of the relationships between sensors. This static graph serves as a complementary component to the dynamically learned graph structure discussed in Section IV-C, providing a robust foundation for capturing inherent sensor dependencies.

C. Graph Structure Learner

Graph structure learner obtains a suitable graph A^{FPM} for anomaly detection task by jointly training with the downstream task. It consists of two main parts: the fully parameterized graph structure learner and the DTW prior graph. Firstly, the fully parameterized graph structure learner treats each element of the adjacency matrix as an independent and learnable parameter. In addition, the graph learning process is guided by the DTW distance prior graph A^{DTW} . This guidance allows the model to incorporate meaningful structural information, improving its ability to model complex spatio-temporal dependencies.

1) *Fully parameterized graph structure learner*: It is a flexible learner and treats the elements in adjacency matrix as free learnable parameters. These parameters are updated through backpropagation, enabling the generation of a customized and optimal graph structure tailored to the specific requirements of downstream tasks. However, since adjacency matrices are typically binary (composed of 0 s and 1 s), gradient-based optimization becomes challenging.

To address this, the Gumbel-Softmax technique [48] is employed to convert discrete parameters into continuous ones, enabling backpropagation. The Gumbel-Softmax trick is used to sample from a categorical distribution while maintaining differentiability. The process of Gumbel-Softmax is defined as follows:

$$g = -\log(-\log(u)), u \sim \text{Uniform}(0, 1), \quad (8)$$

$$z_1^{i,j} = \frac{\exp\left(\left(\log \pi_1^{i,j} + g^{i,j}\right) / \tau\right)}{\sum_{v \in \{0,1\}} \exp\left(\left(\log \pi_v^{i,j} + g^{i,j}\right) / \tau\right)}, \quad (9)$$

where u represents the sample extracted from a $\text{Uniform}(0,1)$ distribution, $g^{i,j}$ follows a Gumbel distribution, and $\pi_1^{i,j}$ denotes the values of the i -th row and j -th column of the probability matrix $\pi_1 \in R^{K \times K}$ which signifies the probability that node i is connected to node j in the graph. Here, τ serves as a temperature parameter. As τ approaches 0, $z_1^{i,j}$ tends towards 0 or 1, making the distribution more discrete.

Ultimately, the i -th row and j -th column in the adjacency matrix $A_{i,j}^{FPM}$ are assigned the value of $z_1^{i,j}$, causing $A_{i,j}^{FPM}$ to be set to 1 with the probability of $\pi_1^{i,j}$.

2) *DTW prior graph*: Due to delay correlation in time series, there may be noise in the prior graph by the Euclidean distance [49]. Compared with the Euclidean distance and cosine distance, DTW distance can capture delay correlations and ensure invariance to time shifts. Therefore, according to Ref. [32],

we utilize the DTW distance [47] to construct prior graph and guide the graph learning process.

Given two time series $Y = (y_1, y_2, \dots, y_n)$ and $Z = (z_1, z_2, \dots, z_m)$, a cost matrix $D \in R^{n \times m}$ is introduced, where $D_{i,j} = |y_i - z_j|$.

$$D_c(i, j) = D_{i,j} + \min(D_c(i-1, j), D_c(i, j-1), D_c(i-1, j-1)), \quad (10)$$

$$\text{DTW}(Y, Z) = D_c(n, m), \quad (11)$$

where D_c is the cumulative cost matrix computed by dynamic programming.

The DTW distance is used to compute the similarity matrix $S^{DTW} \in R^{N \times N}$ for all pairs of sensors:

$$S_{i,j}^{DTW} = \text{DTW}(x^i, x^j), \quad (12)$$

where x^i, x^j represent the time series of sensors i and j , respectively.

The adjacency matrix of DTW prior graph A^{DTW} is constructed by thresholding the similarity matrix:

$$A_{i,j}^{DTW} = \begin{cases} 1, & S_{i,j}^{DTW} < \varepsilon \\ 0, & \text{otherwise} \end{cases}, \quad (13)$$

where ε is a parameter to determine the sparsity of adjacency matrix.

D. STGODE Predictor

Traditional GCNs adopt a discrete formulation, where information is aggregated from n -order neighbors using n layers. However, such discrete GCNs face two significant limitations: (1) they suffer from over-smoothing when stacking multiple layers, and (2) they struggle to effectively capture long-distance dependencies due to limited receptive fields.

To address these issues, we employ the STGODE model [46] as the predictor. Unlike traditional Spatio-Temporal GCNs, STGODE leverages temporal convolutional networks and a continuous GCN formulation based on ODEs. This continuous formulation allows the model to capture long-range spatio-temporal dependencies more effectively. The STGODE predictor operates on both the learned graph and the static similarity graph, combining the advantages of dynamically learned structural information and stable, knowledge-driven priors to enhance prediction accuracy and robustness.

1) *Neural ODEs*: Neural ODEs [41] consider a continuous-time (depth) model,

$$x(t) = x(0) + \int_0^t \frac{dx}{d\tau} d\tau = x(0) + \int_0^t f(x(\tau), \tau) d\tau, \quad (14)$$

where $f(x(\tau), \tau)$ will be parameterised by a neural network to model the hidden dynamic. We can backpropagate the process through an ODE solver without any internal operations, which allows to build it just as a block for the whole neural network.

2) *Continuous GCN with ODE*: Inspired by Neural ODEs, the continuous GCN with ODE can be represented in tensor

formate [50] as follows:

$$\mathcal{H}(s) = \int_0^{s+1} \mathcal{H}_0 \times_1 \hat{A}^\tau \times_2 U^\tau \times_3 W^\tau d\tau, \quad (15)$$

$$\hat{A} = \frac{\alpha}{2}(I + D^{-\frac{1}{2}}AD^{-\frac{1}{2}}), \quad (16)$$

where $\mathcal{H}_0$ denotes the initial input, $\times_i$ denotes the tensor matrix multiplication on mode i , W and U are learnable parameter matrices, and s is a continuous variable of layer n in discrete GCN. D is the degree matrix of A , $\hat{A}$ is the regularized adjacency matrix, and $\alpha \in (0, 1)$ is a hyperparameter. The ODE formulation allows the model to capture long-distance dependencies and avoid over-smoothing by integrating information continuously across layers. The ODE can get an analytical solution by the ODESolver.

3) *STGODE structure*: A STGODE block [46] consists of two temporal convolutional networks (TCN) layers and an ODE-solver with a “sandwich” structure. It can extract spatio-temporal features and capture long-distance dependencies.

$$H_{tcn}^l = \begin{cases} X_t & , l = 0 \\ \sigma(W^l *_{d^l} H_{tcn}^{l-1}) & , l = 1, 2, \dots, L \end{cases}, \quad (17)$$

$$\mathcal{H}(s) = \text{ODESolver}\left(\frac{d\mathcal{H}(s)}{ds}, \mathcal{H}_0, s\right), \quad (18)$$

$$\begin{aligned} \frac{d\mathcal{H}(s)}{ds} &= \mathcal{H}(s) \times_1 (\hat{A} - I) + \mathcal{H}(s) \times_2 (U - I) \\ &+ \mathcal{H}(s) \times_3 (W - I) + \mathcal{H}_0, \end{aligned} \quad (19)$$

where X_t is the input of TCN, H_{tcn}^l is the output of the l -th layer of TCN, W^l denotes the l -th convolution kernel, $d^l = 2^{l-1}$ denotes exponential dilation rate.

To take full advantage of known dependency and learned dependency, we design a STGODE predictor with hybrid graph learning (two kinds of graphs) for predicting future behavior: the static similarity graph and the learned graph. The input of the STGODE predictor consists of the above static and learned adjacency matrices A^{COS} , A^{FPM} and the multivariate time series subsequence X_t . After passing two STGODE layers, max-pooling aggregates information. Finally, the output is fed into a multilayer perceptron (MLP) layer to obtain the final single-step prediction result ($\hat{x}_t$).

E. Loss Function

The total loss function includes two components: the prediction loss and the graph learning loss.

The prediction loss $loss_p$ measures the discrepancy between the predicted values and the ground truth for the MTS prediction task. It is defined as the average absolute error (MAE) across all sensors:

$$loss_p = \frac{1}{K} \sum_{i=1}^K |\hat{x}_t^i - x_t^i|, \quad (20)$$

where $\hat{x}_t^i$ and x_t^i represent the prediction and the ground truth of the i -th sensor at timestamp t , severally.

To enhance the caliber of the fully parameterized graph structure, we use DTW prior graph to guide the graph learning. The graph learning loss $loss_g$ is introduced to improve the quality of the fully parameterized graph structure (A^{FPM}) by incorporating the DTW distance graph (A^{DTW}). It is defined as the cross-entropy between the two adjacency matrices:

$$\begin{aligned} loss_g &= \sum_{ij} -A_{i,j}^{DTW} \log A_{i,j}^{FPM} - (1 - A_{i,j}^{DTW}) \\ &\times \log (1 - A_{i,j}^{FPM}). \end{aligned} \quad (21)$$

The total loss function combines the prediction loss and the graph learning loss, weighted by a regularization parameter.

$$loss = loss_p + \lambda_1 loss_g, \quad (22)$$

where paramete λ_1 is the regularization amplitude parameter that controls the influence of the graph learning loss on the total loss.

F. Anomaly Scores and Threshold Selection

The purpose of anomaly detection is to identify the abnormal situation that deviates from the normal behavior by comparing the ground truth and prediction. Initially, anomaly scores are computed separately for every sensor. Subsequently, these individual scores are aggregated to derive the anomaly scores corresponding to each specific timestamp. Once the anomaly score at timestamp t exceeds the predetermined threshold, the system flags the corresponding timestamp as anomalous.

For each sensor i at timestamp t , the prediction error $Err_i(t)$ is calculated as the absolute difference between the ground truth (x_t^i) and the predicted value ($\hat{x}_t^i$):

$$Err_i(t) = |x_t^i - \hat{x}_t^i| \quad (23)$$

This error quantifies how much the predicted value deviates from the true observed value. Larger errors indicate a potential anomaly.

To maintain uniformity among sensors with varying value scales, the prediction errors undergo a standardization procedure. This normalization rescales the errors into a standard metric using the mean (μ_i) and standard deviation (σ_i) of the errors for sensor i :

$$s_i(t) = \frac{Err_i(t) - \mu_i}{\sigma_i} \quad (24)$$

This step ensures that sensors with larger ranges do not dominate anomaly detection, enabling fair comparisons between sensors.

To compute the overall anomaly score at a given timestamp t , the system selects the maximum normalized error across all sensors:

$$s(t) = \max_i s_i(t) \quad (25)$$

This approach assumes that the most anomalous sensor at a given timestamp determines the timestamp's overall anomaly score.

Grid search technology is pivotal for determining the optimal threshold. In grid search, the upper and lower bounds of the threshold are defined as the maximum and minimum values of $s(t)$, respectively. All possible thresholds are exhaustively

searched with a step size of 0.01. For each threshold, the resulting predictions are compared to the ground truth to compute the F1 score. The threshold yielding the highest F1 score is selected as the optimal threshold. In addition, we use the point adjustment strategy for anomaly scores to improve anomaly detection accuracy.

G. Complexity Analysis

In the MAD-ODE framework, the construction of graph structures plays a critical role in determining the performance and efficiency of the model. Therefore, we analyze the time complexity of generating the static similarity graph, the DTW distance graph, and the learned graph.

- *Static Similarity Graph*: The static similarity graph is constructed based on the cosine similarity between nodes (indicators). For a pair of sensors with a sequence length of M , the time complexity of cosine similarity is $O(M)$. For a graph with N nodes, the cosine similarity must be computed for each pair of nodes. Since there are $N \times N$ node pairs, the time complexity of generating the static similarity graph is $O(N^2 M)$.
- *DTW Distance Graph*: The DTW distance graph is constructed by calculating the DTW similarity between pairs of sensors. For a pair of sensors with a sequence length of M , the time complexity of the DTW distance is $O(M^2)$ due to the need for dynamic programming to compute the cumulative cost matrix. For $N \times N$ sensor pairs, the overall time complexity for constructing the DTW distance graph is $O(N^2 M^2)$.
- *Learned Graph*: The fully parameterized graph structure treats each element of the adjacency matrix as a freely learnable parameter. The adjacency matrix is of size $N \times N$, and during training, the parameters are updated iteratively. The time complexity of updating these parameters is proportional to the number of parameters, resulting in a time complexity of $O(N^2)$ for fully parameterized graph structure learning.

Combining these three components, the overall time complexity of generating the graph structures in the MAD-ODE framework is dominated by the DTW distance graph and is $O(N^2 M^2)$.

V. EXPERIMENT AND PERFORMANCE ANALYSIS

A. Datasets

In this section, five publicly available real-world datasets are utilized for evaluating the performance of the anomaly detection framework. Key statistics about these datasets are summarized in Table I. All the training sets only include normal time series instances. Below is a detailed description of each dataset:

The Secure Water Treatment (SWaT) dataset originates from a water treatment test bench supervised by the Singapore Public Utilities Bureau. This dataset captures 11 days of continuous 24-hour operation, during which network traffic and data from 51 sensors and actuators were recorded. The first 7 days represent

TABLE I
DATASET DESCRIPTION

Datasets	SWaT	WADI	MSL	SMAP	SMD
#Indicators	51	127	55	25	38
#Training	49500	76297	58317	135183	73729
#Testing	45000	17280	73729	427617	708420
#Anomaly rate	11.97%	5.99%	10.72%	13.13%	4.16%

normal operations, while the remaining 4 days include multiple attack scenarios.

The Water Distribution (WADI) dataset is an extension of the SWaT platform and represents a more comprehensive water treatment, storage, and distribution. This dataset contains data from 127 sensors and actuators, collected over 16 days. Of these, 14 days represent normal operations, while the remaining 2 days simulate attack scenarios, making it suitable for anomaly detection tasks.

The Mars Science Laboratory (MSL) dataset includes 55 indicators from 27 entities on NASA's Mars rover, while the Soil Moisture Active Passive (SMAP) dataset contains 25 indicators from 55 entities, representing soil and telemetry data collected by NASA. Each dataset consists of a training subset and a testing subset, with anomalies in both testing subsets explicitly labeled. The sample interval for both datasets is set to 1 minute. To preserve privacy, the timestamp information has been removed. In the MSL dataset, the training sequences of 27 entities range from 179 to 4048, while the testing sequences span from 1096 to 6100. Similarly, in the SMAP dataset, the training sequence lengths of 55 entities vary between 52 and 2621, and the testing sequence lengths range from 7244 to 8640.¹

The Server Machine Dataset (SMD) is a 5-week-long dataset from a large Internet company collected and made publicly available [51]. It contains data from 28 server machines each one monitored by 38 metrics. SMD is divided into two subsets of equal size: the first half is the training set and the second half is the testing set.

For the SWaT and WADI datasets, raw data is down-sampled to a time resolution of 10 seconds. The median value within each 10-second interval is extracted to ensure robustness against noise. Anomalies in the SWaT and WADI datasets are tagged based on ground truth labels provided in the datasets.

B. Experimental Setup

1) *Metrics*: To evaluate the performance of the proposed method and compare it with baseline methods, three key metrics are employed: precision (Prec), recall (Rec), and the F1 score (F1). These metrics are defined as follows:

$$\text{Prec} = \frac{TP}{TP + FP}, \quad (26)$$

$$\text{Rec} = \frac{TP}{TP + FN}, \quad (27)$$

$$F1 = 2 * \frac{\text{Prec} * \text{Rec}}{\text{Prec} + \text{Rec}}, \quad (28)$$

¹<https://github.com/khundman/telemanom>

where TP, TN, FP, and FN are the number of true positives, true negatives, false positives, and false negatives.

2) *Baselines*: The proposed method is compared with 16 baseline methods, including machine learning and deep learning approaches, to assess its effectiveness. Below is a summary of these methods:

- *Autoencoder (AE)*: It reconstructs the input data using an autoencoder and uses the reconstruction error as the anomaly score.
- *Isolation Forest (IF)*: It is a tree-based anomaly detection algorithm that effectively identifies anomalies or outliers in datasets that differ significantly from the majority.
- *DAGMM [40]*: This method combines deep autoencoders with Gaussian mixture models to capture the underlying distribution of normal data and identify anomalies based on reconstruction errors.
- *LSTM-NDT [34]*: This method utilizes LSTM to capture contextual information and dependencies in sequential data for anomaly detection.
- *LSTM-VAE [38]*: This method projects multimodal observations and time dependencies into a latent space and reconstructs the expected distribution using an LSTM-based VAE.
- *MAD-GAN [36]*: This method uses LSTM as the underlying model in a GAN framework to capture the time series data's temporal correlation.
- *OmniAnomaly [39]* aims to learn the normal representation of time series data, and then calculate reconstruction errors to effectively identify anomalous points that deviate from the normal patterns.
- *USAD [52]*: This method trains an encoder-decoder framework in an adversarial manner to achieve fast and efficient training.
- *MTAD-GAT [11]*: This method treats the relationships between indicators as a complete graph and employs a GAT to detect anomalies.
- *GDN [19]*: This method constructs a graph structure using pairwise cosine similarities between nodes, utilizes GAT to learn dependencies between time series and perform prediction.
- *FuSAGNet [21]*: This method learns the graph structure using the pairwise cosine similarity between recursive sensor embeddings. It obtains a sparse representation of the input data by a sparse autoencoder and integrates it into a GAT to perform sensor behavior prediction.
- *GTA [27]*: This method automatically learns the graph structure through a direct approach and employs graph convolution combined with a transformer-based architecture to capture temporal dependencies.
- *MGCLAD [29]*: This method achieves anomaly detection by concurrently learning intra- and inter-signal graph structures, capturing temporal context and dependencies between signals. It performs anomaly detection by comparing inputs and reconstructing outputs.
- *MEGLAD [30]*: This method employs an alternate multiple graph structure learning approach to capture sensor relationships from various perspectives, minimizing

TABLE II
METHOD PARAMETERS

Parameter	Value
Sliding Window Size ω	12
Graph Loss Regularization Parameter λ_1	1
Learning Rate	0.005
Batch Size	64
Epochs	30
The number of neighbors in static graph k	10
The similarity threshold in DTW prior graph ϵ	0.6

information loss. Additionally, it uses an extended prior graph generating method to adaptively select neighbors by enhancing the weights of highly correlated nodes.

- *FuGLAD [31]*: This method employs three kinds of typical graph structure learners to capture as many relationship types among sensors as possible and exploits the prior similarity to evaluate the importance of all learned graphs and adaptively learn the fusion weight instead of the direct average weight.
- *MSTGAD [32]*: MSTGAD uses dynamic time warping distance as prior knowledge to effectively capture delay correlations and designs an ensemble predictor to ensure high prediction accuracy and stable anomaly scores.

3) *Experimental Parameter Setting*: The experimental parameter configurations are detailed in Table II. All experiments were conducted using Python 3.8, PyTorch 1.10, and CUDA version 11.3, on a server equipped with an Intel (R) Xeon (R) Platinum 8255c CPU and an NVIDIA RTX 4090 GPU. According to Ref. [19], the k in static graph is set to 10, window size w is set to 12. According to Ref. [32], the ϵ in DTW distance graph is set to 0.6.

4) *Baseline Method Parameter Setting*: For the baseline methods, we set the sliding window size ω in GDN and FuSAG-Net to 5, and GTA to 60. The baseline is configured according to the source code or open-source code.

C. Detection Performance and Efficiency

As shown in Table III, our method achieves either the best or the second-best performance on individual datasets and consistently achieves the best average performance across all datasets. This demonstrates the robustness and generalization capability of our approach in handling diverse anomaly detection tasks. While MGCLAD demonstrates the best performance on the WADI dataset, its performance is significantly weaker on the other four datasets. Similarly, MSTGAD achieves superior results on the SWaT and SMAP datasets but performs poorly on WADI. In contrast, our method introduces a hybrid graph learning approach that incorporates a static graph structure to enhance the robustness of anomaly detection models. By leveraging the STGODE predictor, our approach effectively captures long-distance dependencies between sensors, producing more representative predictions. This design not only improves the model's adaptability to diverse datasets but also ensures stable and accurate anomaly detection performance across varying scenarios. To filter false positives and enhance precision, our

TABLE III
PRECISION, RECALL, AND F1 SCORE ON THE DATASETS

Method	SWaT			WADI			SMAP			MSL			SMD			F1-average	#1 st	#2 nd
	Prec	Rec	F1	Prec	Rec	F1	Prec	Rec	F1	Prec	Rec	F1	Prec	Rec	F1			
AE	72.63	52.63	61.03	34.35	34.35	34.35	72.16	97.95	77.76	85.35	97.48	87.92	88.25	80.37	82.80	68.77	0	0
IF	96.2	73.15	83.11	62.41	61.55	61.98	44.23	51.05	46.71	56.81	67.4	59.84	59.38	85.32	58.66	62.06	0	0
DAGMM	27.46	69.52	39.37	54.44	26.99	36.09	63.34	99.84	71.24	75.62	98.03	81.12	67.30	84.50	72.31	60.03	0	0
LSTM-NDT	77.78	51.09	61.67	1.38	78.23	2.71	85.23	73.26	78.79	62.88	100	77.21	97.36	84.4	90.42	62.16	0	0
LSTM-VAE	96.24	59.91	73.85	87.79	14.45	24.82	71.64	98.75	75.55	85.99	97.56	85.37	75.76	90.08	82.3	68.38	0	0
MAD-GAN	98.97	63.74	77.54	41.44	33.92	37.3	81.57	92.16	86.54	85.16	99.3	91.69	97.79	81.44	88.87	76.39	0	0
OmniAnomaly	72.23	98.32	83.28	26.52	97.99	41.74	75.85	97.56	85.35	91.4	88.91	90.14	83.68	86.82	85.22	77.15	0	0
USAD	100	56	71.79	43.09	22.51	29.57	74.8	96.27	84.19	79.49	99.12	88.22	93.14	96.17	93.82	73.52	0	0
MTAD-GAT	21.03	64.46	31.71	11.72	30.55	16.94	79.91	99.91	88.8	79.17	98.24	87.68	82.10	92.15	86.83	62.39	0	0
GDN	99.35	68.12	80.82	97.5	40.19	56.92	74.8	98.91	85.18	93.08	98.92	95.91	69.36	71.14	70.30	77.83	0	0
FuSAGNet	98.7	72.6	83.7	82.9	47.8	60.7	9.3	67.9	16.5	80.6	98.9	88.9	68.73	73.21	70.90	64.14	0	0
GTA	93.9	85.7	89.6	79.6	79.4	79.5	89.11	91.76	90.41	91.04	91.17	91.11	88.72	83.24	85.89	87.30	0	0
MGCLAD	94.81	89.11	91.87	98	87.84	92.64	93.42	92.71	93.07	95.1	95.89	95.49	95.17	93.52	94.34	93.48	1	0
MEGLAD	96.14	92.51	94.29	87.34	77.92	82.36	91.55	100	95.59	99.44	100	99.72	89.31	91.32	90.30	92.45	0	0
FuGLAD	97.59	91.69	94.55	86.47	85.44	85.95	94.44	100	97.14	99.93	100	99.96	93.55	96.34	94.92	94.50	0	2
MSTGAD	98.36	93.31	95.77	86.2	84.5	85.3	94.6	100	97.2	99.8	100	99.9	99.96	100	<u>99.98</u>	<u>95.63</u>	2	<u>1</u>
MAD-ODE	96.87	93.1	94.94	83.96	90.19	86.96	94.4	100	97.14	99.93	100	99.97	100	100	100	95.80	2	3

TABLE IV
PERFORMANCE AND EFFICIENCY ACROSS DATASETS

Dataset	AUC	AUPR	Epoch Time(s)	Inference Time(s)
SWaT	0.9807	0.7832	53.8	0.015
WADI	0.9941	0.7175	119.8	0.051
SMAP	0.9632	0.8606	3.3	0.010
MSL	0.9998	0.9999	1.4	0.019
SMD	1.0	1.0	25.7	0.011

model could be further processed using density-aware clustering or voxel-based neighborhood analysis [53], [54].

The performance of all methods on the WADI dataset is lower compared to other datasets. This is because the WADI dataset has a longer sequence length, more indicators, and a lower anomaly rate, as shown in Table I. However, on the WADI dataset, with the help of the static similarity graph structure and the graph structure learning module, we enrich and uncover dependencies between sensors. By leveraging long-distance dependency relationships captured by the STGODE-based predictor, our method continues to demonstrate superior performance compared to other baseline methods. This demonstrates that our method is effective in handling high-dimensional time series and scenarios with sample imbalance, making it suitable for practical applications.

The performance metrics, Area Under the Curve (AUC) and Area Under the Precision-Recall Curve (AUPR), are critical for evaluating the predictive accuracy and precision of the model without the need for manual threshold setting. These metrics provide a robust and threshold-independent assessment of model performance, making them particularly suitable for anomaly detection tasks with imbalanced datasets. As shown in Table IV, the results show that all datasets achieve high AUC and AUPR values, indicating strong model performance across different data characteristics.

As shown in Table IV, the training time required for all datasets is remarkably short, with a maximum of 30 epochs. Specifically, the MSL dataset has the shortest training time, while the WADI dataset requires the longest training time. Nevertheless, even for WADI, the training time of epoch is only 119.8 seconds. This efficiency in training is highly advantageous for experimental workflows and iterative model development,

significantly reducing the time required for experimentation and enabling faster progress. We also include the inference time of our system in Table IV to provide a clear measurement of the overall detection latency, i.e., the time from anomaly onset to alarm generation. The most time-consuming component of the system is the ODEs module. This is due to the computational complexity involved in solving the equations to model dependencies in the data.

D. Ablation Experiment

To validate the effectiveness of the hybrid graph learning (the static and learned graph structures), we conduct ablation experiments. First, the STGODE predictor is evaluated using only the static graph (Only COS) or only the learned graph (Only FPM). Additionally, we introduce a self-adaptive graph learning structure (ADP) [55] to replace one of these components. ADP+COS indicates that the STGODE predictor is supported by both the static graph and the self-adaptive graph learning structure. Similarly, ADP+FPM means that the STGODE predictor is supported by both the fully parameterized learned graph and the self-adaptive graph learning structure.

The adjacency matrix of the self-adaptive graph is defined as follows:

$$A^{ADP} = \text{SoftMax}(\text{ReLU}(E_1 E_2^T)), \quad (29)$$

where $E_1, E_2 \in R^{N \times e}$ represent two node embedding dictionaries initialized with learnable parameters.

The experimental results are presented in Table V. Using only the static graph or the fully parameterized graph results in a decline in model performance. This demonstrates that both graph structures are indispensable and play a critical role in the model's performance. Moreover, when either of these graph structures is replaced with the self-adaptive graph learning structure, the model's performance decreases. This further confirms the effectiveness of our proposed static graph structure and fully parameterized graph structure.

TABLE V
ABLATION EXPERIMENT

Method	Prec	SWaT Rec	F1	Prec	WADI Rec	F1	Prec	SMAP Rec	F1	Prec	MSL Rec	F1	Prec	SMD Rec	F1
Only COS	99.17	82.71	90.19	61.92	96.93	75.57	73.76	100	84.90	99.75	100	99.87	100	100	100
Only FPM	89.03	84.88	86.91	53.87	85.44	66.07	81.92	100	90.07	99.93	100	<u>99.96</u>	99.73	100	99.86
ADP+COS	94.18	89.38	91.72	63.76	97.92	<u>77.23</u>	78.48	100	87.94	99.93	100	<u>99.96</u>	99.88	100	99.94
ADP+FPM	99.06	87.2	<u>92.75</u>	64.41	91.18	75.49	91.89	100	<u>95.77</u>	99.87	100	99.93	99.92	100	<u>99.96</u>
MAD-ODE	96.87	93.1	94.94	83.96	90.19	86.96	94.4	100	97.14	99.93	100	99.97	100	100	100

TABLE VI
PERFORMANCE WITH NOISE AND MISSING VALUES

Method	Prec	SWaT Rec	F1	Prec	WADI Rec	F1	Prec	SMAP Rec	F1	Prec	MSL Rec	F1	Prec	SMD Rec	F1
MAD-ODE	96.87	93.1	94.94	83.96	90.19	86.96	94.4	100	97.14	99.93	100	99.97	100	100	100
Gaussian noise	89.86	93.86	91.82	78.06	90.19	83.69	92.11	100	95.89	100	100	100	100	100	100
Miss values 1%	95.81	90.93	93.31	77.66	90.19	83.46	94.44	100	97.14	99.87	100	99.93	100	100	100
Miss values 2%	95.57	90.82	93.13	76.87	90.19	83.01	92.01	100	95.83	99.87	100	99.93	99.96	100	99.98
Miss values 5%	93.11	91.13	92.11	73.29	90.19	80.86	87.89	100	93.55	99.93	100	99.96	100	100	100

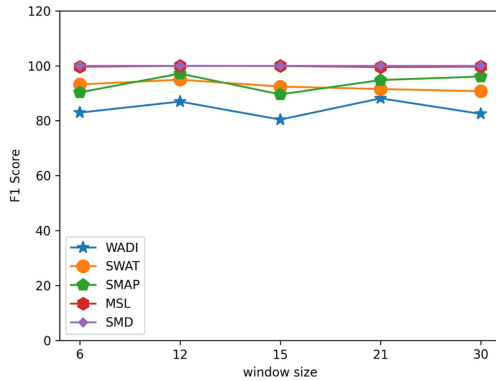


Fig. 4. The impact of window size.

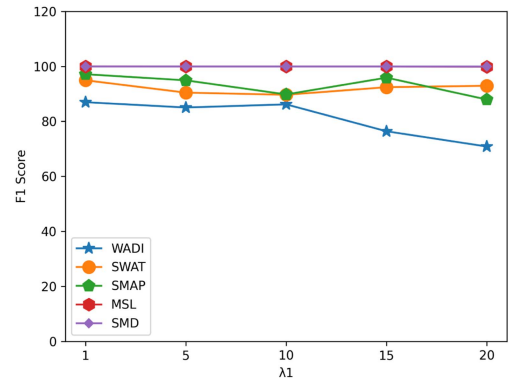


Fig. 5. The impact of prediction weight parameter.

E. Parameter Influence

To evaluate the stability of our method under different parameter settings, we analyze the impact of hyperparameters on model performance.

1) *Window Size*: In this experiment, we set the window size w to 6, 12, 15, 21, and 30. The performance of models with different window sizes is shown in Fig. 4.

For the SMAP, MSL and SMD datasets, the window size has a negligible impact on the detection results. However, for the SWaT and WADI datasets, increasing the window size tends to degrade the model's performance. This is because larger window sizes may obscure short anomalies, making accurate detection more challenging.

2) *Graph Loss Regularization Parameter*: We analyze the impact of the graph loss regularization parameter λ_1 , which is set to 1, 5, 10, 15, and 20, across the five datasets. The experimental results are shown in Fig. 5. Excessively large values of λ_1 can lead to overfitting in the graph structure learning, negatively affecting the model's performance. On the other hand, smaller values of λ_1 tend to yield more stable experimental results. Therefore, it is recommended to choose smaller values for the graph loss regularization parameter.

F. Robustness

To address concerns regarding the system's performance under real-world conditions, we conduct experiments to evaluate the robustness of our anomaly detection framework. Specifically, we introduce the following perturbations to simulate practical challenges. Gaussian noise with a standard deviation of 0.1 is added to the sensor data to simulate measurement inaccuracies. Portions of the data are randomly removed at rates of 1%, 2%, and 5% to mimic sensor outages or communication failures.

As shown in Table VI, the results demonstrate that the performance of our framework is largely unaffected by noise and missing data in the MSL and SMD datasets. However, a slight performance degradation is observed in the SWaT, WADI, and SMAP datasets. These results highlight the robustness of our framework under typical real-world perturbations.

G. Case Study

As shown in Fig. 6, we conduct a case study to reveal the meaning of the learned graph structure. Fig. 6 (Left) presents a subset of the graph structure learned by our method, while Fig. 6 (Right) displays the predicted results of the corresponding sensor values. Specifically, during the timestamps from 1180

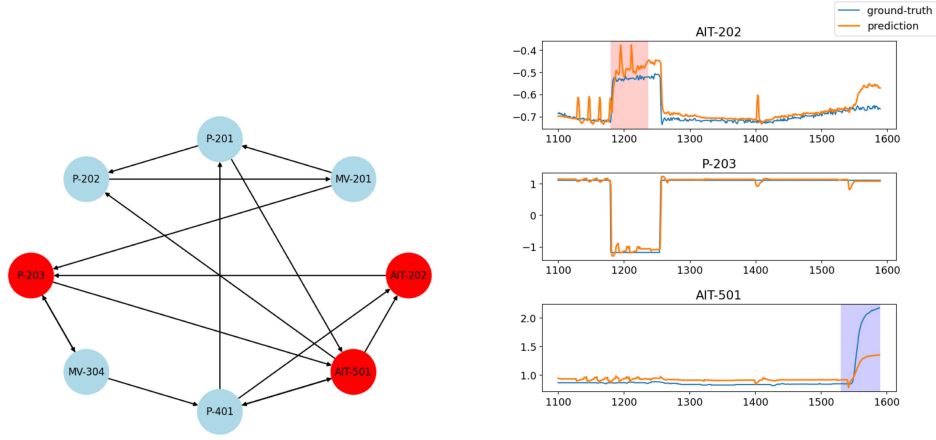


Fig. 6. **Left:** Partial graph structure learned on SWaT dataset. **Right:** Understand the relationship between sensors with ground truth and prediction.

to 1237, sensor AIT-202 was subjected to an attack, which we highlighted in red in the graph. In the water treatment process, sensors interact with one another. Due to these interactions, the attack on AIT-202 caused the dosing pump P203 to cease operation. As the subsequent water treatment steps progressed, the permeability conductivity analyzer AIT-501 was ultimately affected at timestamp 1530, as shown in Fig. 6 (Right).

Our model accurately predicts the changes in AIT-501, demonstrating its effectiveness. Additionally, Fig. 6 (Left) precisely captures the dependency relationships among the three sensors involved in this case study.

VI. CONCLUSION AND FUTURE WORK

In this work, we propose the multivariate time series anomaly detection method based on learnable STGODE. Our approach employs an STGODE-based predictor with hybrid graph learning, which integrates both the static graph structure and the fully parameterized learned graph structure. This hybrid design enables the model to effectively capture long-distance dependencies among sensors, accurately forecast future sensor behaviors, and significantly improve the F1 score. Compared to baseline methods, MAD-ODE demonstrates superior and stable performance in detecting anomalies across multivariate time series data. By leveraging the complementary strengths of static and learnable graph structures, MAD-ODE ensures both robustness and adaptability in complex IoT environments. In future work, a promising direction is to incorporate false-positive filtering mechanisms as a post-processing step. We aim to extend this approach to diagnose anomalies in IoT systems, enabling a deeper understanding of their root causes and further enhancing system reliability.

REFERENCES

- [1] S. He et al., "A joint matrix factorization and clustering scheme for irregular time series data," *Inf. Sci.*, vol. 644, 2023, Art. no. 119220.
- [2] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *Proc. 25th Annu. Netw. Distrib. Syst. Secur. Symp.*, San Diego, CA, USA, Feb. 2018, pp. 1–15 [Online]. Available: https://www.ndss-symposium.org/wp-content/uploads/2018/02/ndss2018_03A-3_Mirsky_paper.pdf
- [3] I. J. King and H. H. Huang, "Euler: Detecting network lateral movement via scalable temporal graph link prediction," in *Proc. 29th Annu. Netw. Distrib. Syst. Secur. Symp.*, San Diego, CA, USA, Apr. 2022, pp. 1–36. [Online]. Available: <https://www.ndss-symposium.org/ndss-paper/auto-draft-227/>
- [4] C. Fu, Q. Li, and K. Xu, "Detecting unknown encrypted malicious traffic in real time via flow interaction graph analysis," in *Proc. 30th Annu. Netw. Distrib. Syst. Secur. Symp.*, San Diego, CA, USA, Feb./Mar. 2023, pp. 1–18 [Online]. Available: <https://www.ndss-symposium.org/ndss-paper/detecting-unknown-encrypted-malicious-traffic-in-real-time-via-flow-interaction-graph-analysis/>
- [5] M. Ike, K. Phan, K. Sadoski, R. Valme, and W. Lee, "Scaphy: Detecting modern ICS attacks by correlating behaviors in SCADA and physical," in *Proc. 44th IEEE Symp. Secur. Privacy*, San Francisco, CA, USA, May 2023, pp. 20–37, doi: [10.1109/SP46215.2023.10179411](https://doi.org/10.1109/SP46215.2023.10179411).
- [6] S. He, Z. Li, J. Wang, and N. N. Xiong, "Intelligent detection for key performance indicators in industrial-based cyber-physical systems," *IEEE Trans. Ind. Informat.*, vol. 17, no. 8, pp. 5799–5809, Aug. 2021.
- [7] J. Liu et al., "A generic framework for finding special quadratic elements in data streams," *IEEE/ACM Trans. Netw.*, vol. 32, no. 4, pp. 3269–3284, Aug. 2024.
- [8] S. He et al., "Fine-grained multivariate time series anomaly detection in iot," *Comput., Mater. Continua*, vol. 75, no. 3, pp. 5027–5047, May 2023.
- [9] J. Zhou et al., "Graph neural networks: A review of methods and applications," *AI Open*, vol. 1, pp. 57–81, 2020.
- [10] Q. He, Y. Zheng, C. Zhang, and H.-Y. Wang, "MTAD-TF: Multivariate time series anomaly detection using the combination of temporal pattern and feature pattern," *Complexity*, vol. 2020, pp. 1–9, 2020.
- [11] H. Zhao et al., "Multivariate time-series anomaly detection via graph attention network," in *Proc. IEEE Int. Conf. Data Mining*, 2020, pp. 841–850.
- [12] D. Scheinert, A. Acker, L. Thamsen, M. K. Geldenhuys, and O. Kao, "Learning dependencies in distributed cloud applications to identify and localize anomalies," in *Proc. IEEE/ACM Int. Workshop Cloud Intell.*, 2021, pp. 7–12.
- [13] L. Zhou, Q. Zeng, and B. Li, "Hybrid anomaly detection via multihead dynamic graph attention networks for multivariate time series," *IEEE Access*, vol. 10, pp. 40967–40978, 2022.
- [14] W. Chen, L. Tian, B. Chen, L. Dai, Z. Duan, and M. Zhou, "Deep variational graph convolutional recurrent network for multivariate time series anomaly detection," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 3621–3633.
- [15] W. Zhang, C. Zhang, and F. Tsung, "GRELEN: Multivariate time series anomaly detection from the perspective of graph relational learning," in *Proc. 31st Int. Joint Conf. Artif. Intell.*, 2022, pp. 2390–2397.
- [16] Y. Zhu et al., "A survey on graph structure learning: Progress and opportunities," 2021, *arXiv:2103.03036*.
- [17] D. Chai, L. Wang, and Q. Yang, "Bike flow prediction with multi-graph convolutional networks," in *Proc. 26th ACM SIGSPATIAL Int. Conf. Adv. Geograph. Inf. Syst.*, 2018, pp. 397–400.
- [18] H. Pang et al., "Asymptotic consistent graph structure learning for multivariate time series anomaly detection," *IEEE Trans. Instrum. Meas.*, vol. 73, 2024, Art. no. 2509510.

- [19] A. Deng and B. Hooi, "Graph neural network-based anomaly detection in multivariate time series," in *Proc. AAAI Conf. Artif. Intell.*, 2021, vol. 35, pp. 4027–4035.
- [20] R. Gao, W. He, L. Yan, D. Liu, Y. Yu, and Z. Ye, "Hybrid graph transformer networks for multivariate time series anomaly detection," *J. Supercomput.*, vol. 80, no. 1, pp. 642–669, 2024.
- [21] S. Han and S. S. Woo, "Learning sparse latent graph representations for anomaly detection in multivariate time series," in *Proc. 28th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2022, pp. 2977–2986.
- [22] W. Zhang, P. He, C. Qin, F. Yang, and Y. Liu, "A graph attention network-based model for anomaly detection in multivariate time series," *J. Supercomput.*, vol. 80, no. 6, pp. 8529–8549, 2024.
- [23] J. Wang, S. Shao, Y. Bai, J. Deng, and Y. Lin, "Multiscale wavelet graph autoencoder for multivariate time-series anomaly detection," *IEEE Trans. Instrum. Meas.*, vol. 72, 2023, Art. no. 2502911.
- [24] S. He, G. Li, J. Wang, K. Xie, and P. Sharma, "UNI-directional graph structure learning-based multivariate time series anomaly detection with dynamic prior knowledge," *Int. J. Mach. Learn. Cybern.*, vol. 16, pp. 267–283, May 2024.
- [25] Z. Zhang, Z. Geng, and Y. Han, "Graph structure change-based anomaly detection in multivariate time series of industrial processes," *IEEE Trans. Ind. Informat.*, vol. 20, no. 4, pp. 6457–6466, Apr. 2024.
- [26] H. Pang et al., "Asymptotic consistent graph structure learning for multivariate time-series anomaly detection," *IEEE Trans. Instrum. Meas.*, vol. 73, 2024, Art. no. 2509510.
- [27] Z. Chen, D. Chen, X. Zhang, Z. Yuan, and X. Cheng, "Learning graph structures with transformer for multivariate time-series anomaly detection in iot," *IEEE Internet Things J.*, vol. 9, no. 12, pp. 9179–9189, Jun. 2022.
- [28] X. Zhou, C. Dai, W. Wang, and T. Qiu, "Global-local association discrepancy for multivariate time series anomaly detection in IIoT," *IEEE Internet Things J.*, vol. 11, no. 7, pp. 11287–11297, Apr. 2024.
- [29] S. Qin, L. Chen, Y. Luo, and G. Tao, "Multiview graph contrastive learning for multivariate time-series anomaly detection in IoT," *IEEE Internet Things J.*, vol. 10, no. 24, pp. 22401–22414, Dec. 2023.
- [30] S. He, G. Li, Q. Guo, and K. Xie, "Multi-graph structure learning-based multivariate time series anomaly detection with extended prior knowledge," in *Proc. 27th Int. Conf. Comput. Supported Cooperative Work Des.*, 2024, pp. 109–114.
- [31] S. He, G. Li, K. Xie, and P. K. Sharma, "Fusion graph structure learning-based multivariate time series anomaly detection with structured prior knowledge," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 8760–8772, 2024.
- [32] S. He, Q. Guo, G. Li, K. Xie, and P. K. Sharma, "Multivariate time series anomaly detection based on multiple spatio-temporal graph convolution," *IEEE Trans. Instrum. Meas.*, vol. 74, 2025, Art. no. 3500714.
- [33] S. He et al., "Graph structure learning-based multivariate time series anomaly detection in Internet of Things for human-centric consumer applications," *IEEE Trans. Consum. Electron.*, vol. 70, no. 3, pp. 5419–5431, Aug. 2024.
- [34] K. Hundman, V. Constantinou, C. Laporte, I. Colwell, and T. Soderstrom, "Detecting spacecraft anomalies using LSTMS and nonparametric dynamic thresholding," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2018, pp. 387–395.
- [35] C. Zhang et al., "A deep neural network for unsupervised anomaly detection and diagnosis in multivariate time series data," in *Proc. AAAI Conf. Artif. Intell.*, 2019, vol. 33, pp. 1409–1416.
- [36] D. Li, D. Chen, B. Jin, L. Shi, J. Goh, and S.-K. Ng, "MAD-GAN: Multivariate anomaly detection for time series data with generative adversarial networks," in *Proc. Int. Conf. Artif. Neural Netw.*, 2019, pp. 703–716.
- [37] K.-J. Jeong, J.-D. Park, K. Hwang, S.-L. Kim, and W.-Y. Shin, "Two-stage deep anomaly detection with heterogeneous time series data," *IEEE Access*, vol. 10, pp. 13704–13714, 2022.
- [38] D. Park, Y. Hoshi, and C. C. Kemp, "A multimodal anomaly detector for robot-assisted feeding using an LSTM-based variational autoencoder," *IEEE Robot. Automat. Lett.*, vol. 3, no. 3, pp. 1544–1551, Jul. 2018.
- [39] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, and D. Pei, "Robust anomaly detection for multivariate time series through stochastic recurrent neural network," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2019, pp. 2828–2837.
- [40] B. Zong et al., "Deep autoencoding Gaussian mixture model for unsupervised anomaly detection," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–19.
- [41] T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud, "Neural ordinary differential equations," in *Proc. 31st Annu. Conf. Adv. Neural Inf. Process. Syst.*, S. Bengio, H. M. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, Eds., Montréal, Canada, Dec. 2018, pp. 6572–6583.
- [42] J. H. Fririksson and E. Ågren, "Neural ordinary differential equations for anomaly detection," Master Degree Thesis, Kungliga Tekniska högskolan, Jan. 4, 2022.
- [43] H. Lim, S. Park, M. J. K. Lee, S. Lim, and N. Park, "MADSGM: Multivariate anomaly detection with score-based generative models," in *Proc. 32nd ACM Int. Conf. Inf. Knowl. Manage.*, I. Frommholz, F. Hopfgartner, M. M. Lee, M. Oakes, M. L. Zhang, and R. L. T. Santos, Eds., Birmingham, U.K., Oct. 2023, pp. 1411–1420.
- [44] R. Hu, X. Yuan, Y. Qiao, B. Zhang, and P. Zhao, "Unsupervised anomaly detection for multivariate time series using diffusion model," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process.*, Seoul, Republic of Korea, Apr. 2024, pp. 9606–9610.
- [45] Y. Liu, Y. Zheng, D. Zhang, H. Chen, H. Peng, and S. Pan, "Towards unsupervised deep graph structure learning," in *Proc. ACM Web Conf.*, Lyon, France, Apr. 2022, pp. 1392–1403, doi: [10.1145/3485447.3512186](https://doi.org/10.1145/3485447.3512186).
- [46] Z. Fang, Q. Long, G. Song, and K. Xie, "Spatial-temporal graph ODE networks for traffic flow forecasting," in *Proc. 27th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, Virtual Event, Singapore, Aug. 2021, pp. 364–373.
- [47] H. Sakoe and S. Chiba, "Dynamic programming algorithm optimization for spoken word recognition," *IEEE Trans. Acoust., Speech, Signal Process.*, vol. ASSP-26, no. 1, pp. 43–49, Feb. 1978.
- [48] E. Jang, S. Gu, and B. Poole, "Categorical reparameterization with gumbel-softmax," in *Proc. 5th Int. Conf. Learn. Representations*, Toulon, France, Apr. 2017, pp. 24–26. [Online]. Available: <https://openreview.net/forum?id=rkE3y85ee>
- [49] K. Choi, J. Yi, C. Park, and S. Yoon, "Deep learning for anomaly detection in time-series data: Review, analysis, and guidelines," *IEEE Access*, vol. 9, pp. 120043–120065, 2021.
- [50] L. A. C. Xhonneux, M. Qu, and J. Tang, "Continuous graph neural networks," in *Proc. 37th Int. Conf. Mach. Learn.*, Jul. 2020, vol. 119, pp. 10432–10441.
- [51] Y. Su, C. Zhao, R. N. Liu, W. Sun, and D. Pei, "Robust anomaly detection for multivariate time series through stochastic recurrent neural network," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, A. V. Teredesai, Y. K. Li, R. Rosales, E. Terzi, and G. Karypis, Eds., Anchorage, AK, USA, Aug. 2019, pp. 2828–2837.
- [52] J. Audibert, P. Michiardi, F. Guyard, S. Marti, and M. A. Zuluaga, "USAD: Unsupervised anomaly detection on multivariate time series," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2020, pp. 3395–3404.
- [53] B. A. AlAhmadi, L. Axon, and I. Martinovic, "99% false positives: A qualitative study of SOC analysts' perspectives on security alarms," in *Proc. 31st USENIX Secur. Symp.*, K. R. Butler and K. Thomas, Eds., Boston, MA, USA, Aug. 2022, pp. 2783–2800. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity22/presentation/alahmadi>
- [54] C. Fu, Q. Li, K. Xu, and J. Wu, "Point cloud analysis for ML-based malicious traffic detection: Reducing majorities of false positive alarms," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, W. Meng, C. D. Jensen, C. Cremers, and E. Kirda, Eds., Copenhagen, Denmark, Nov. 2023, pp. 1005–1019, doi: [10.1145/3576915.3616631](https://doi.org/10.1145/3576915.3616631).
- [55] Z. Wu, S. Pan, G. Long, J. Jiang, and C. Zhang, "Graph wavenet for deep spatial-temporal graph modeling," in *Proc. 28th Int. Joint Conf. Artif. Intell.*, S. Kraus, Ed., Macao, China, Aug. 2019, pp. 1907–1913.



Shiming He received the BS degree in information security and the PhD degree in computer science and technology from Hunan University, China, in 2006 and 2013, respectively. She is currently a Professor with the School of Computer Science and Technology, Changsha University of Science and Technology, Changsha, China. Her research interests include machine learning, data analysis, AIOps, and anomaly detection.



Qingqing Guo received the BS degree from Hunan Agricultural University, in 2022. He is currently working toward the MS degree in computer technology with the Changsha University of Science and Technology. His research interests include deep learning, data analysis, and anomaly detection.



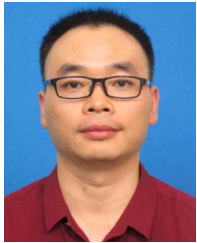
Kun Xie (Member, IEEE) received the PhD degree in computer application from Hunan University, Changsha, China, in 2007. She has authored or coauthored more than 100 articles in major journals and conference proceedings, including journals such as *IEEE/ACM Transactions on Networking*, *IEEE Transactions on Mobile Computing*, *IEEE Transactions on Computers*, *IEEE Transactions on Parallel and Distributed Systems*, *IEEE Transactions on Wireless Communications*, and *IEEE Transactions on Services Computing*, and conferences, including SIGMOD, INFOCOM, ICDCS, SECON, DSN, and IWQoS. Her research interests include cover network measurement, network security, Big Data, and AI.



Keyao Feng received the BS degree from East China Jiaotong University in 2023. She is currently working toward the MS degree in software engineering with the Changsha University of Science and Technology. Her research interests include deep learning, graph structure learning, and anomaly detection.



Pradip Kumar Sharma (Senior Member, IEEE) received the PhD degree in CSE from the Seoul National University of Science and Technology, South Korea, in 2019. He was a postdoctoral research fellow in the Department of Multimedia Engineering with the Dongguk University, South Korea. He was a software engineer with MAQ Software, India, and involved on variety of projects, proficient in building largescale complex data warehouses, OLAP models, and reporting solutions that meet business objectives and align IT with business. He is currently an assistant professor of Cybersecurity in the Department of Computing Science with the University of Aberdeen, U.K. He has authored or coauthored many technical research papers in leading journals from IEEE, Elsevier, Springer, and MDPI. Some of his research findings are authored or coauthored in the most cited journals. His current research interests are focused on the areas of Cybersecurity, Blockchain, Edge computing, SDN, and IoT security. He has been an expert reviewer for IEEE TRANSACTIONS, Elsevier, Springer, and MDPI journals and magazines. He is listed in the world's Top 2% scientists for citation impact during the calendar year 2019 by Stanford University, Stanford, CA, USA. He has also been invited to serve as the technical programme committee member and the chair in several reputed international conferences such as IEEE DASC 2021, IEEE CNCC 2021, CSA 20202, IEEE ICC2019, IEEE MENACOMM'19, and 3ICT 2019. He is also an associate editor for *Peer-to-Peer Networking and Applications*, *Human-centric Computing and Information Sciences*, *Electronics* (MDPI), and *Journal of Information Processing Systems* journals. He is a guest editor of international journals of certain publishers such as IEEE, Elsevier, Springer, MDPI, and JIPS. He was the recipient of the top 1% reviewer in computer science by Publons Peer Review Awards 2018 and 2019, Clarivate Analytics.



Diqing Liang received the PhD degree in computer software and theory from Central South University, China, in 2017. He is currently a Senior Engineer with the Information Construction Management Department of the Changsha University of Science & Technology, Changsha, China. His research interests include artificial intelligence, Big Data, and cyberspace security.